SOFTWARE REQUIREMENTS SPECIFICATION

University Management System (ICT Education)

Document Version: 2.0.0

Status: Final

Classification: Academic Project Submission

Revision History

Version	Date	Description	Author
0.1	2026-07-03	Initial requirements baseline derived from `docs/product_proposal.pdf`	ICT Education Dev
1.0	2026-07-04	Requirement Traceability Matrix established; FR-054–FR-056 added during Project Readiness Audit	ICT Education Dev
1.1–1.9	2026-07-05 – 2026-07-23	Requirements refined and verified milestone-by-milestone (M0–M10)	ICT Education Dev
2.0	2026-07-24	Final release — all 12 milestones (M0–M11) complete and approved, `v2.0.0` tagged	ICT Education Dev

---

Table of Contents

---

1. Introduction

1.1. Purpose

This Software Requirements Specification (SRS) describes the functional and non-functional requirements of the University Management System (ICT Education), a web platform that consolidates university operations — attendance, exams, results, fees, and scheduling — into a single system. It is intended for academic evaluation and as a permanent reference for the system's delivered scope. This document reflects the **actual, implemented, and tested system** as of release `v2.0.0`; no described capability is speculative or planned-but-unbuilt unless explicitly marked under Section 11 (Future Enhancements).

1.2. Scope

The system serves four roles — **Student**, **Teacher**, **Admin**, and **Parent** — through a decoupled architecture: a React 18 + TypeScript single-page application (SPA) frontend communicating with a FastAPI (Python 3.12) REST API backend, backed by a PostgreSQL database. In scope: authentication and role-based access control, user/profile management, reference data (departments, courses, rooms, semesters), scheduling and timetabling, attendance tracking, examinations and grading, results and transcripts, fee management, notifications, role-specific dashboards, and administrative reporting. Out of scope: payment gateway integration, mobile native applications, and any capability listed in Section 11.

1.3. Definitions, Acronyms, and Abbreviations

Term	Meaning
API	Application Programming Interface
SPA	Single Page Application
RBAC	Role-Based Access Control
JWT	JSON Web Token
ORM	Object-Relational Mapper
CRUD	Create, Read, Update, Delete
FR	Functional Requirement
NFR	Non-Functional Requirement
BR	Business Rule

Term	Meaning
VR	Validation Rule
GPA	Grade Point Average
CI	Continuous Integration
CDN	Content Delivery Network

1.4. References

- `docs/product_proposal.pdf` — original project specification
- `docs/Requirement_Analysis.md` — FR-001–FR-056, NFR-001–NFR-016
- `docs/System_Architecture.md` — architecture, auth/authz flows, folder structure
- `docs/Database_Design.md` — full schema, relationships, constraints
- `docs/API_Contract.md` — full REST API documentation
- `docs/UI_Wireframes.md` — text wireframes for every screen
- `docs/Requirement_Traceability_Matrix.md` — every requirement traced to tables/APIs/pages/status
- `docs/Implementation_Roadmap.md` — milestone-by-milestone build order
- `docs/Proposal_vs_Engineering_Additions.md` — every engineering addition beyond the original proposal
- `CHANGELOG.md`, `PROJECT_PROGRESS.md` — full implementation history

---

2. Overall Description

2.1. Product Perspective

The system is a new, standalone product — it does not integrate with or replace any pre-existing university software. It replaces informal processes (spreadsheets, email-distributed results, disconnected finance tools) with a single, role-aware platform. The frontend and backend are independently deployable: the SPA is built to static assets servable from a CDN or any static host (e.g. via `nginx`, see `docker/Dockerfile.frontend`), and the API is a containerized service (see `docker/Dockerfile.backend`) that can be scaled independently.

2.2. Product Functions

At a high level, the system provides:

- Secure authentication and session management (JWT access/refresh tokens)
- Role-based access control enforced at the API layer for all 68 endpoints
- Student, Teacher, Parent, and Admin account lifecycle management
- Department, course, room, and semester reference-data management
- Class scheduling, timetables, and conflict detection
- Attendance marking, correction, and automatic low-attendance warnings
- Exam creation (MCQ/written/coding/mixed), timed exam-taking, and grading
- Result submission → approval → publication workflow, with PDF transcripts
- Fee structure definition, payment recording, invoicing, and overdue tracking

- Automatic and manual notifications across four event types
- Role-specific dashboards (Student, Teacher, Parent, Admin)
- Administrative reporting (attendance, results, fees) by department/semester/student

2.3. User Classes and Characteristics

Role	Description
Student	Primary end user. Views own exams, attendance, results, fees, timetable, and notifications. No create/update/delete permissions anywhere.
Teacher	Manages the academic side of assigned classes: exam creation/grading, attendance marking, result submission, timetable change requests.
Admin	System controller: user account lifecycle, result approval, fee structures, timetable publishing, and all reporting.
Parent	Read-only access to a linked child's attendance, results, and fee status, via a manually-supplied student identifier (see Section 11 known limitation on a dedicated child-selector screen).

2.4. Operating Environment

- **Backend:** Python 3.12, FastAPI, served by Uvicorn (development) or Uvicorn/Gunicorn workers in a container (production-style, see `docker/Dockerfile.backend`)
- **Database:** PostgreSQL (developed and tested against PostgreSQL 16)
- **Frontend:** Any modern evergreen browser (Chromium, Firefox, Edge, Safari) supporting ES2020
- **Minimum runtime dependencies:** Node.js 20 (frontend build), Python 3.12 (backend runtime)

2.5. Assumptions

- A single PostgreSQL instance serves the whole application; no read replicas or sharding are assumed.
- The institution operates in a single time zone context for scheduling purposes (per `Database_Design.md`).
- The low-attendance warning threshold is 80%, resolved as a documented engineering assumption (`Requirement_Analysis.md` §14 item 4) since the source proposal did not specify a number.
- Password complexity policy is a minimum length of 8 characters; the proposal does not specify a stronger policy (`Requirement_Analysis.md` §14 item 13).

2.6. Dependencies

The backend depends on PostgreSQL being reachable via `DATABASE_URL` and a configured `JWT_SECRET_KEY`. The frontend depends on the backend API being reachable via `VITE_API_BASE_URL`/`VITE_API_ROOT_URL`. Full dependency lists are pinned in `backend/requirements.txt` and `frontend/package.json`.

---

3. Functional Requirements

Each subsection below reflects the actual endpoints and business rules implemented and tested in the system. Requirement IDs (FR-xxx) trace to `docs/Requirement_Analysis.md` and `docs/Requirement_Traceability_Matrix.md`.

3.1. Authentication (FR-001–FR-005)

- Login with email/password issues a short-lived JWT access token and a longer-lived refresh token (`POST /auth/login`).
- An expired access token can be silently refreshed without re-entering credentials (`POST /auth/refresh`), using single-active-session refresh-token rotation.
- Logout invalidates the current session's refresh token server-side (`POST /auth/logout`).
- Any authenticated user can change their own password (`PUT /auth/password`).
- Login redirects to a role-specific dashboard.
- `POST /auth/login` is rate-limited (5 attempts per 60 seconds per client IP) as a security hardening measure added in Milestone 11.

3.2. Role-Based Access Control

- Every one of the 68 endpoints enforces role-only checks via a `require_roles(*roles)` dependency, and ownership/linkage checks (a Student's own data, a Parent's linked child) are re-verified in the service layer on every request — never trusted to the frontend alone.
- A deactivated account (`user.is_active = false`) fails authorization immediately, even with an otherwise-valid token.
- Resources outside a caller's scope return `404 Not Found` rather than `403 Forbidden`, so their existence is never leaked.

3.3. Students (FR-009–FR-013, part of FR-006–FR-008)

- Admin/Teacher can list all students (`GET /users/students`, paginated, department-filterable).
- Admin can create (`POST /users/students`), retrieve (`GET /users/students/{id}`, Admin/Teacher), update (`PUT /users/students/{id}`), and deactivate (`DELETE /users/students/{id}`, soft delete — `is_active = false`, never a row deletion) a student account.
- A Student can retrieve and update their own profile (`GET`/`PUT /users/me`).

3.4. Teachers (FR-014–FR-016)

- Admin can list all teachers (`GET /users/teachers`), create a teacher account (`POST /users/teachers`), and update a teacher's record (`PUT /users/teachers/{id}`).

3.5. Parents

- Parent accounts and the `parent_student_link` M:N relationship exist in the schema and are used for ownership verification across attendance, results, and fees endpoints (a `student_id` query parameter is verified against the caller's actual linked children on every request). No account-creation endpoint or a "list my linked children" endpoint exists — this is a documented, permanent limitation (see Section 11).

3.6. Departments and Courses (reference data)

- List/create/get-by-id endpoints exist for `Department`, `Course`, `Room`, and `Semester` (`/departments`, `/courses`, `/rooms`, `/semesters`). Reads are open to any authenticated role; creates are Admin-only. These are foundational reference data underpinning student/teacher assignment, scheduling, and fee-structure scoping.

3.7. Attendance (FR-026–FR-032)

- A Student views their own attendance summary, filterable by class/date, with a live-computed overall percentage (`GET /attendance/me`).

- A Teacher marks attendance for a class and date (`POST /attendance`), and a Teacher/Admin can retrieve (`GET /attendance/{classId}`) or correct (`PUT /attendance/{id}`) a record.
- Admin generates attendance reports by department or semester (`GET /attendance/reports`).
- A low-attendance warning (below 80%) is issued automatically, once, on the threshold-crossing event.
- A Parent can view a linked child's attendance (via the same `GET /attendance/{classId}`, Parent-scoped).

3.8. Timetable / Scheduling (FR-045–FR-051)

- A Student/Teacher views their own timetable (`GET /schedule/me`).
- Admin creates, updates, and removes schedule entries (`POST`/`PUT`/`DELETE /schedule/{id}`), and detects double-booking conflicts before publishing (`GET /schedule/conflicts`).
- A Teacher requests a timetable change; Admin resolves it (`POST /schedule/change-requests`, `POST /schedule/change-requests/{id}/resolve`).
- A schedule change notifies both enrolled students and the assigned Teacher.

3.9. Exams (FR-017–FR-025)

- Any role lists exams relevant to them (`GET /exams`); a Teacher creates an exam with MCQ/short-answer/descriptive/coding questions, per-question marks, and a time limit (`POST /exams`).
- A Student starts (`POST /exams/{id}/start`, server-clock-only timing) and submits (`POST /exams/{id}/submit`) an exam within its time limit.
- A Teacher grades a submission (`POST /exams/{id}/grade`), awarding marks per question up to the question's maximum, with optional feedback; grading is a re-saveable action.
- A Teacher/Admin retrieves all results for an exam (`GET /exams/{id}/results`); a Student sees per-question marks only once results are published (BR-001 masking).
- An exam can be deleted only while unpublished (`DELETE /exams/{id}`).

3.10. Results (FR-033–FR-037)

- A Student views their own results and per-semester GPA across all semesters (`GET /results/me`), credit-hour-weighted.
- A Teacher submits graded results for Admin approval (`POST /results/{examId}/submit`); Admin approves or rejects (`POST /results/{id}/approve`, rejection requires a comment).
- A Student or Admin downloads an official PDF transcript (`GET /results/{studentId}/transcript`).
- A Parent views a linked child's published results (same `GET /results/me`, Parent-scoped via `student_id`).
- Only `published` results are ever visible to Students/Parents — an approval workflow gate (NFR-014).

3.11. Fees (FR-038–FR-044, FR-056) — Optional module, fully implemented

- A Student/Parent retrieves fee status and payment history (`GET /fees/me`).
- Admin defines a fee structure per department/semester (`POST /fees`), which automatically generates one invoice per eligible enrolled student.
- Admin records a payment (`POST /fees/payments`); overpayment beyond the outstanding balance is strictly disallowed.
- Admin/Parent retrieves payment history for a student (`GET /fees/payments/{studentId}`); a Student/Admin downloads a PDF invoice (`GET /fees/invoices/{id}`).

- Admin lists overdue accounts (`GET /fees/overdue`) and manually triggers an overdue notice, individually or in bulk (`POST /fees/overdue/notify`).
- An automatic `fee_due` notification fires at invoice-issuance time.

3.12. Notifications (FR-051–FR-053)

- Four automatic triggers: `result_published`, `schedule_change`, `attendance_warning`, `fee_due` — each dispatched synchronously immediately after its originating transaction commits.
- Any authenticated user views their own notification feed with read/unread state (`GET /notifications`) and marks a notification read (`PUT /notifications/{id}/read`).

3.13. Reports (FR-030, FR-054, FR-055)

- Admin generates attendance reports (`GET /attendance/reports`), result reports — grade distribution, pass/fail counts, average GPA (`GET /results/reports`) — and fee reports — collected/outstanding/overdue totals (`GET /fees/reports`) — each filterable by department, semester, or student.

3.14. Dashboards (part of FR-005, UI-level)

- **Student Dashboard:** Upcoming Exams, Attendance %, Fee Status, Recent Results.
- **Teacher Dashboard:** Classes Today, Pending Grading count (computed client-side from existing exam data).
- **Parent Dashboard:** Fee Status and Recent Results (via a manually-supplied linked-child identifier); Attendance % and Upcoming Exams show an honest "Not available" state (no backing endpoint exists for these, by design — see Section 11).
- **Admin Dashboard:** Pending Result Approvals, Overdue Fees, Recent User Signups, and a link to the Reports page.

3.15. User Management (FR-009–FR-016) and Profile (FR-006–FR-008)

See Sections 3.3–3.4 above. Profile management (`GET`/`PUT /users/me`) is available to every role; a Student's own academic history is accessible via `GET /results/me` but is not additionally surfaced on the Profile screen itself (FR-008, a documented permanent limitation — see Section 11).

---

4. Non-Functional Requirements

ID	Requirement	How it is satisfied
NFR-001	RBAC enforced at the API layer for every endpoint	`require_roles()` dependency + service-layer ownership checks on all 68 endpoints
NFR-002	A Student can only access their own data	Ownership checks in every `/me`-scoped service method
NFR-003	A Parent can only access their linked child's data	`parent_student_link` verified server-side on every Parent-scoped request
NFR-004	JWT with short-lived access tokens and refresh rotation	15-minute access tokens, rotating refresh tokens, single-active-session model
NFR-005	All endpoints return JSON, namespaced under `/api/v1`	Enforced by the FastAPI app factory's router prefix

ID	Requirement	How it is satisfied
NFR-006	Relational integrity across all domains	Foreign keys, unique constraints, and check constraints on every table
NFR-007	Schema changes via versioned migrations	10 sequential Alembic revisions, single head
NFR-008	SPA with loading/error states for all async operations	Every React Query-backed page has explicit loading and error UI
NFR-009	Near real-time notification delivery	Synchronous dispatch immediately after the originating transaction commits — no queue delay
NFR-010	Type-safe, component-driven frontend	TypeScript strict mode, no `any`, one component per file
NFR-011	Consistent utility-first styling	TailwindCSS exclusively, no separate stylesheet system
NFR-012	Frontend deployable via CDN, decoupled from backend	Static build (`docker/Dockerfile.frontend`) served independently of the API container
NFR-013	Sufficient code documentation	Nine governing design documents plus inline "why, not what" comments throughout
NFR-014	Approval workflow before visibility (results, fees)	`submitted` → `published/rejected` result workflow; overdue status computed, not manually flagged
NFR-015	No double-booking of rooms/teachers	Real interval-overlap conflict detection on schedule create/update
NFR-016	Attendance % and fee status computed on demand	Never cached/stored; recomputed from persisted rows on every request

---

5. Database Design Summary

PostgreSQL, 26 tables across 8 domains, accessed exclusively through the SQLAlchemy ORM:

Domain	Tables
Reference data	`department`, `course`, `room`, `semester`
Identity & roles	`user`, `student`, `teacher`, `parent`, `admin`, `parent_student_link`
Scheduling	`class_session`, `enrollment`, `schedule_entry`, `schedule_change_request`
Attendance	`attendance_record`
Exams & grading	`exam`, `question`, `question_option`, `exam_submission`, `answer`, `question_grade`
Results	`result`
Fees	`fee_structure`, `invoice`, `payment`

Domain	Tables
Notifications	`notification`

Full column-level design, indexes, and constraints are documented in `docs/Database_Design.md`. Managed by 10 Alembic migrations (`0001`–`0010`); current head `0010`.

---

6. REST API Summary

68 REST endpoints, versioned under `/api/v1`, grouped by router: `auth` (4), `users` (10), `reference_data` (12), `schedule` (10), `attendance` (5), `exams` (10), `results` (5), `fees` (7), `notifications` (2), `reports` (2), plus `health` (1, unversioned). Full request/response contracts, validation rules, and error codes are documented in `docs/API_Contract.md`. Every error response follows a single consistent JSON envelope: `{ "error": { "code", "message", "details" } }`.

---

7. System Architecture

```
Browser (React SPA)<br/>      |  HTTPS / 
;JSON / /api/v1/*<br/>      v<br/>FastAPI Application<b
r/>  Router -> Service -> Repository -> SQLAlchemy ORM<
br/>  Middleware: JWT auth, RBAC, rate limiting, error 
handling, request logging<br/>      |<br/>   
   v<br/>PostgreSQL Database (26 tables)
```

Routers shape HTTP request/response only; Services own all business rules, validation, and RBAC/ownership checks; Repositories are the sole location of SQLAlchemy queries. This layering is enforced without exception across all 11 backend domains. Full detail in `docs/System_Architecture.md`.

---

8. Technology Stack

Layer	Technology
Frontend framework	React 18 + TypeScript
Frontend build tool	Vite
Styling	TailwindCSS
API data layer (frontend)	React Query (TanStack Query)
HTTP client	Axios
Backend framework	FastAPI (Python 3.12)
Database	PostgreSQL
ORM	SQLAlchemy 2.0
Migrations	Alembic
Authentication	JWT (`python-jose`), bcrypt password hashing
PDF generation	ReportLab
Testing (backend)	pytest
Testing (frontend)	Vitest, React Testing Library
Linting (frontend)	ESLint (flat config)

Layer	Technology
Containerization	Docker (`docker-compose` for local dev)
CI	GitHub Actions

---

9. Testing Summary

Suite	Count	Notes
Backend unit tests	—	Service-layer business rules, repositories stubbed
Backend integration tests	—	Full request → DB → response cycle against a disposable PostgreSQL database
Backend total	**349**	All passing
Frontend component tests	**7**	Exam timer/auto-submit, grading form + validation error, result approval/reject-with-comment workflow

Every backend business rule (BR-xxx) and validation rule (VR-xxx) has at least one corresponding test. RBAC and ownership checks have explicit negative tests (wrong-role/wrong-owner rejection). Both CI workflows (`backend-ci.yml`, `frontend-ci.yml`) run the full respective suite on every push/PR.

---

10. Deployment

- **Local development:** `docker compose up` (from `docker/`) starts PostgreSQL, the backend with hot reload, and the Vite dev server.
- **Production-style images:** `docker/Dockerfile.backend` (Uvicorn, pinned dependencies) and `docker/Dockerfile.frontend` (static build served by `nginx`) build independently deployable images.
- **Database migrations** run via `alembic upgrade head` as a deployment step, before the new API version serves traffic.
- **CI:** GitHub Actions verifies both the backend (against a disposable Postgres service container) and frontend on every push/PR.

---

11. Future Enhancements

The following are realistic, scoped improvements — not currently implemented, and explicitly out of scope for the delivered `v2.0.0` release:

- Redis-backed (or otherwise distributed) rate limiting, to replace the current in-memory/single-process login rate limiter for horizontally-scaled deployments.
- Email notification delivery, in addition to the existing in-app notification feed.
- SMS notification delivery for time-sensitive alerts (attendance warnings, fee due dates).
- A production-oriented Docker Compose profile (e.g. Gunicorn workers, managed Postgres, reverse proxy/TLS termination).
- Cloud deployment automation (a specific provider was never specified by the source proposal).

- CI/CD enhancements: automated deployment on tag, dependency vulnerability scanning.
- Deeper analytics on the Admin dashboard (trends over time, not just point-in-time snapshots).
- Structured audit-log retrieval UI (audit events are already logged server-side; there is no dedicated screen to browse them).
- A native or hybrid mobile application.
- A dedicated Parent "Child View" screen with a real linked-children selector, which requires a "list my linked children" endpoint that does not currently exist (a permanent, documented limitation of the current release — see `docs/UI_Wireframes.md` §17).

---

12. Conclusion

The University Management System (ICT Education) delivers all 12 planned milestones: authentication and authorization, user and profile management, scheduling, attendance, examinations and grading, results and transcripts, fee management, notifications, role-specific dashboards and reporting, and a final hardening/testing/deployment pass. The system comprises 68 REST endpoints across 26 database tables, verified by 349 backend tests and 7 frontend component tests, with clean static analysis (`tsc`, `eslint`), a clean migration history (single Alembic head, empty autogenerate diff), and CI pipelines covering both halves of the stack. This document, together with the nine governing design documents in `docs/`, constitutes the complete requirements record for release `v2.0.0`.